

Safety Must Preempt—and the Evidence Must Survive

Project Hands-on Day 5: Preempt and Prove the Whole
Chain

`safety_v1` → `runtime_async_v1` → `release_v1`

Mentor – Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors – Mr. Jiang Suyu, Mr. Ruan Haozhe

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

safety_v1 remains the only route to the actuator

Input `safety_v1` $\longrightarrow$ **Output** `release_v1`

Run once from a clean tree

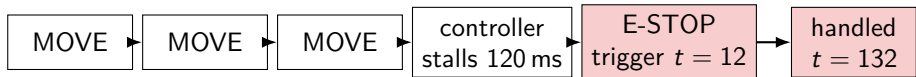
```
make verify-day4
```

Record the commit, compiler, seed, and frozen 50 ms deadline. On failure, restore the latest class-generated `safety_v1` checkpoint.

Freeze before editing

- `safety_thresholds` and verdict values;
- append-only audit fields and single-writer API;
- zero unsafe actuation on all Day 4 fixtures.

A safe FIFO can still deliver emergency work too late



Reproduce one failure: start the supplied overload fixture, stall the controller plug-in, then trigger e-stop while ordinary work occupies the FIFO.

The safety gate still rejects bad motion, but it cannot stop a queue from consuming the deadline.

Safety without timely preemption is still a failed safety property.

Record this signature

- trigger $\rightarrow$ dequeue = 120 ms;
- frozen deadline = 50 ms;
- ordinary work ahead > 0 ;
- verdict = DEADLINE_MISS.

Readiness, deadlines, and cancellation create bounded exits

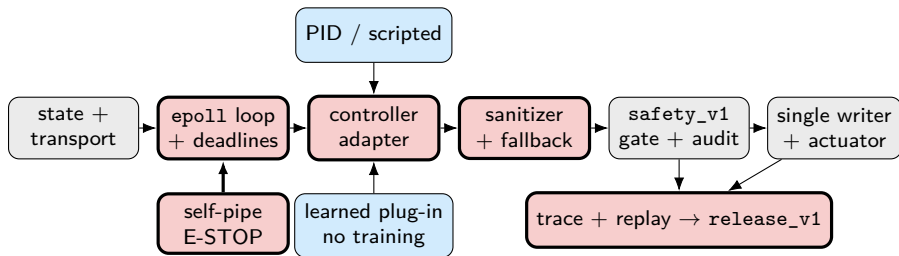
1. **epoll** keeps a persistent fd watch set; use level-triggered mode today.
2. **self-pipe/eventfd** wakes a sleeping loop for e-stop or worker completion.
3. **timerfd** represents controller and absolute e-stop deadlines as readable events.
4. **cancel generation** invalidates queued work and late controller results without killing a thread.

Soda Code

```
fn run(rt: &mut Runtime) -> RtResult<()> {
  loop {
    let events = rt.epoll.wait()?;
    if events.is_ready(WAKE_TOKEN) {
      rt.wake.drain()?;
      rt.cancel_gen.bump();
      rt.motion_queue.clear();
      actuator_submit(estop_cmd())?;
      continue; // preempt
    }
    dispatch(&events, rt.cancel_gen)?;
  }
}
```

Handle e-stop readiness first.

Preemption, gatekeeping, and evidence form one release path

**Change today**

runtime_async, controller adapter, trace propagation, replay index

Frozen from Day 4

thresholds, verdict/reason enum, audit schema, and the only actuator-submit path

Five teams close two chapters through one release gate

Team	Build	Deliver	Pass
G1 1/2	Replay state through the controller host; measure age under load.	Ch1/2 release evidence	State age stays within its limit.
G2 3/4	Preempt MQTT/latest-value work; replay corrupt and stale frames.	Ch3/4 verdict traces	Bad transport input never writes.
G3 5/6	Add epoll, timerfd, wake fd, cancel/watchdog; run 100 stops.	runtime_async_v1 + Ch5/6 data	All 100 stops meet the deadline.
G4 7/8	Connect cancellation to safety, writer, and append-only audit.	Ch7/8 fault matrix	Unsafe writes = 0; audit complete.
G5 9/10	Propagate trace IDs, replay all fixtures, build evidence index.	release_v1 + Ch9/10 index	verify-day5 is green.

Preempt ordinary work, then preserve one complete trace

Soda Code

```

1 let mut trace = TraceCtx::start(state.seq);
2 if events.is_ready(WAKE_TOKEN) {
3     wake.drain()?;
4     cancel_gen = cancel_gen.wrapping_add(1);
5     motion_queue.clear();
6     actuator_submit(estop_cmd()); // still gated
7     trace.record(ESTOP_PREEMPT);
8     continue;
9 }
10 if events.is_ready(CONTROLLER_TOKEN) {
11     let result = controller.read()?;
12     // TODO 1: discard stale generation.
13     // TODO 2: sanitize or choose PID fallback.
14     let cmd = sanitize_or_hold(result)?;
15     gatekeeper_submit(cmd, &mut trace)?;
16     audit_append(&trace)?;
17 }

```

Fixed workload

50 Hz planner + fast/slow/flood
 100 timed e-stops
 stall / NaN / range / low confidence
 six canonical release fixtures

Roles: A—cancel; B—sanitizer/gate; C—trace, replay, and index.

Minute 11: green / blocked / using class checkpoint. **Invariant:** no direct write and no verdict without a trace.

The final release passes only when every path is replayable

Run the canonical gate

```
make verify-day5
```

Save these raw files:

- raw/estop_100.csv
- raw/controller_faults.json
- raw/release_traces.log
- evidence-index.csv

Frozen check	Required result
Happy path	APPROVE; one write; full trace
Corrupt / stale / unsafe	REJECT; zero writes
50 ms e-stop deadline; ordinary ahead	100/100; 0
Stall / NaN / late controller result	FALLBACK or DISCARD
Replay and chapter evidence index	exact verdict; 10/10 linked
Report p50/p95/p99/max; max decides timing. Every chapter claim links to a raw row or trace ID.	

release_v1 becomes the book's reproducible backbone

Input safety_v1 → **Output** release_v1

safety_v1 → runtime_async_v1 → release_v1

- **Release:** source, build manifest, raw logs, fixture seeds, and replay command.
- **Evidence index:** Ch1–Ch10 claim → test, trace, table, or figure.
- **Sign-off:** both chapter leads, Integration/Evidence Lead, and next-team reviewer.

Record the worst-case latency, known limitation, and owner before creating the final tag.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template